

Software Engineering

1.1 Software, Software Engineering as layered approach and its characteristics, Types of software.

1.2 Software development framework.

Software Process Framework.

1.3 Process models; Perspective Process Models, Specialized Process Models.

1.4 Agile Software development: Agile Process and its importance, Extreme Programming, Adaptive Software Development, Scrum, Dynamic Systems Development Method (DSDM), Crystal.

Selection Criteria for software process model

Waterfall, Incremental, RAID, Spiral

Software Engineering is the process of developing a software product in a well-defined systematic approach.

Software engineering is the process of analyzing user needs and then designing, constructing, and testing end-user applications that will satisfy these needs through the use of software programming languages.

Parameters Defining Software Project

The software should be produced at a reasonable cost, in a reasonable time, and should be of good quality. These three parameters often drive and define a software project.

1. **Cost:** As the main cost of producing software is the manpower employed, the cost of developing software is generally measured in terms of **person-months** of effort spent in development. The productivity in the software industry for writing fresh code mostly ranges from a few hundred to about 1000 + LOC per person per month.
2. **Schedule:** The schedule is another important factor in many projects. Business trends are dictating that the time to market a product should be reduced; that is, the cycle time from concept to delivery should be small. This means that software needs to be developed faster and within the specified time.
3. **Quality:** Quality is one of the main mantras, and business strategies are designed around it. Developing high-quality software is another fundamental goal of software engineering.

Attributes of Software

The international standard on software product quality suggests that software quality comprises six main attributes:

1. **Reliability:** The capability to provide failure-free service.
2. **Functionality:** The capability to provide functions that meet stated and implied needs when the software is used.
3. **Usability:** The capability to be understood, learned, and used.
4. **Efficiency:** The capability to provide appropriate performance relative to the amount of resources used.
5. **Maintainability:** the capability to be modified for purposes of making corrections, improvements, or adaptations.
6. **Portability:** The capability to be adapted for different specified environments without applying actions or means other than those provided for this purpose in the product.

Classification of Software

The software can be classified based on various **criteria**, including:

1. **Purpose:** Software can be classified as [system software](#) (e.g., operating systems, device drivers) or [application software](#) (e.g., word processors, games).
2. **Platform:** Software can be classified as native software (designed for a specific operating system) or cross-platform software (designed to run on multiple operating systems).
3. **Deployment:** Software can be classified as installed software (installed on the user's device) or cloud-based software (hosted on remote servers and accessed via the internet).
4. **License:** Software can be classified as proprietary software (owned by a single entity) or open-source software (available for free with the source code accessible to the public).
5. **Development Model:** Software can be classified as traditional software (developed using a waterfall model) or [agile software](#) (developed using an iterative and adaptive approach).
6. **Size:** Software can be classified as small-scale software (designed for a single user or small group) or enterprise software (designed for large organizations).

7. **User Interface:** Software can be classified as [Graphical User Interface \(GUI\)](#) software or [Command-Line Interface \(CLI\)](#) software.

These classifications are important for understanding the characteristics and limitations of different types of software, and for selecting the best software for a particular need.

Types of Software

The software is used extensively in several domains including hospitals, banks, schools, defense, finance, stock markets, and so on. It can be categorized into different types:

1. Based on Application

The software can be classified on the basis of the application. These are to be done on this basis.

1. System Software:

System Software is necessary to manage computer resources and support the execution of application programs. Software like operating systems, compilers, editors and drivers, etc., come under this category. A computer cannot function without the presence of these. [Operating systems](#) are needed to link the machine-dependent needs of a program with the capabilities of the machine on which it runs. [Compilers](#) translate programs from high-level language to machine language.

2. Application Software:

Application software is designed to fulfill the user's requirement by interacting with the user directly. It could be classified into two major categories:- generic or customized. Generic Software is software that is open to all and behaves the same for all of its users. Its function is limited and not customized as per the user's changing requirements. However, on the other hand, customized software is the software products designed per the client's requirement, and are not available for all.

3. Networking and Web Applications Software:

Networking Software provides the required support necessary for computers to interact with each other and with data storage facilities. Networking

software is also used when software is running on a network of computers (such as the World Wide Web). It includes all network management software, server software, security and encryption software, and software to develop web-based applications like [HTML](#), [PHP](#), [XML](#), etc.

4. Embedded Software:

This type of software is embedded into the hardware normally in the Read-Only Memory (ROM) as a part of a large system and is used to support certain functionality under the control conditions. Examples are software used in instrumentation and control applications like washing machines, satellites, microwaves, etc.

5. Reservation Software:

A Reservation system is primarily used to store and retrieve information and perform transactions related to air travel, car rental, hotels, or other activities. They also provide access to bus and railway reservations, although these are not always integrated with the main system. These are also used to relay computerized information for users in the hotel industry, making a reservation and ensuring that the hotel is not overbooked.

6. Business Software:

This category of software is used to support business applications and is the most widely used category of software. Examples are software for inventory management, accounts, banking, hospitals, schools, stock markets, etc.

7. Entertainment Software:

Education and Entertainment software provides a powerful tool for educational agencies, especially those that deal with educating young children. There is a wide range of entertainment software such as computer games, educational games, translation software, mapping software, etc.

8. Artificial Intelligence Software:

Software like expert systems, decision support systems, pattern recognition software, [artificial neural networks](#), etc. come under this category. They involve complex problems which are not affected by complex computations using non-numerical algorithms.

9. Scientific Software:

Scientific and engineering software satisfies the needs of a scientific or engineering user to perform enterprise-specific tasks. Such software is written for specific applications using principles, techniques, and formulae particular to that field. Examples are software like MATLAB, AUTOCAD, PSPICE, ORCAD, etc.

10. Utility Software:

The programs coming under this category perform specific tasks and are different from other software in terms of size, cost, and complexity. Examples are antivirus software, voice recognition software, compression programs, etc.

11. Document Management Software:

Document Management Software is used to track, manage, and store documents to reduce the paperwork. Such systems are capable of keeping a record of the various versions created and modified by different users (history tracking). They commonly provide storage, versioning, metadata, security, as well as indexing and retrieval capabilities.

2. Based on Copyright

Classification of Software can be done based on copyright. These are stated as follows:

1. Commercial Software:

It represents the majority of software that we purchase from software companies, commercial computer stores, etc. In this case, when a user buys software, they acquire a license key to use it. Users are not allowed to make copies of the software. The company owns the copyright of the program.

2. Shareware Software:

Shareware software is also covered under copyright, but the purchasers are allowed to make and distribute copies with the condition that after testing the software, if the purchaser adopts it for use, then they must pay for it. In both of the above types of software, changes to the software are not allowed.

3. Freeware Software:

In general, according to freeware software licenses, copies of the software can be made both for archival and distribution purposes, but here, distribution cannot be for making a profit. Derivative works and modifications to the software are allowed and encouraged. Decompiling of the program code is also allowed without the explicit permission of the copyright holder.

4. Public Domain Software:

In the case of public domain software, the original copyright holder explicitly relinquishes all rights to the software. Hence, software copies can be made both for archival and distribution purposes with no restrictions on distribution. Modifications to the software and reverse engineering are also allowed.

Layered Technology in Software Engineering

[Software engineering](#) is a fully layered technology, to develop software we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer.

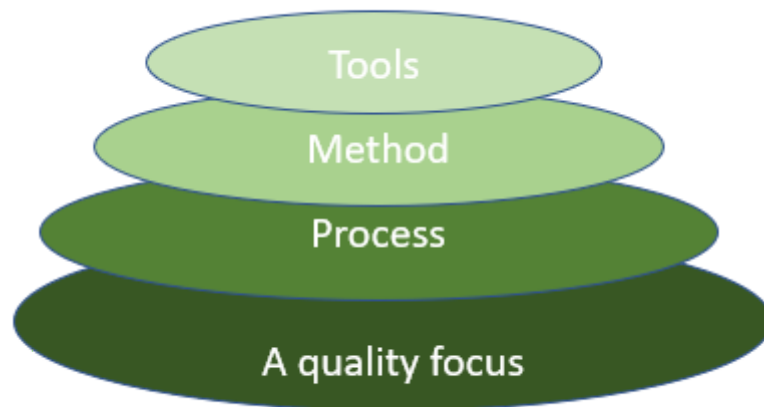


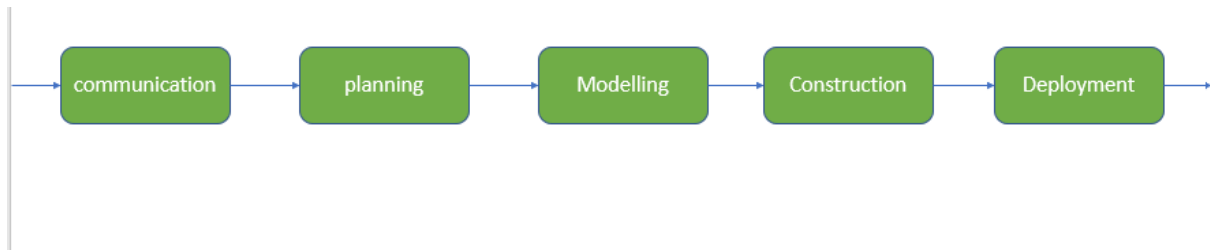
Fig: The diagram shows the layers of software development

Layered technology is divided into four parts:

1. A quality focus: It defines the continuous process improvement principles of software. It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability.

2. Process: It is the foundation or base layer of software engineering. It is key that binds all the layers together which enables the development of software before the deadline or on time.

Process defines a framework that must be established for the effective delivery of software engineering technology. The software process covers all the activities, actions, and tasks required to be carried out for software development.



Process activities are listed below:-

- **Communication:** It is the first and foremost thing for the development of software. Communication is necessary to know the actual demand of the client.
- **Planning:** It basically means drawing a map for reduced the complication of development.
- **Modeling:** In this process, a model is created according to the client for better understanding.
- **Construction:** It includes the coding and testing of the problem.
- **Deployment:-** It includes the delivery of software to the client for evaluation and feedback.

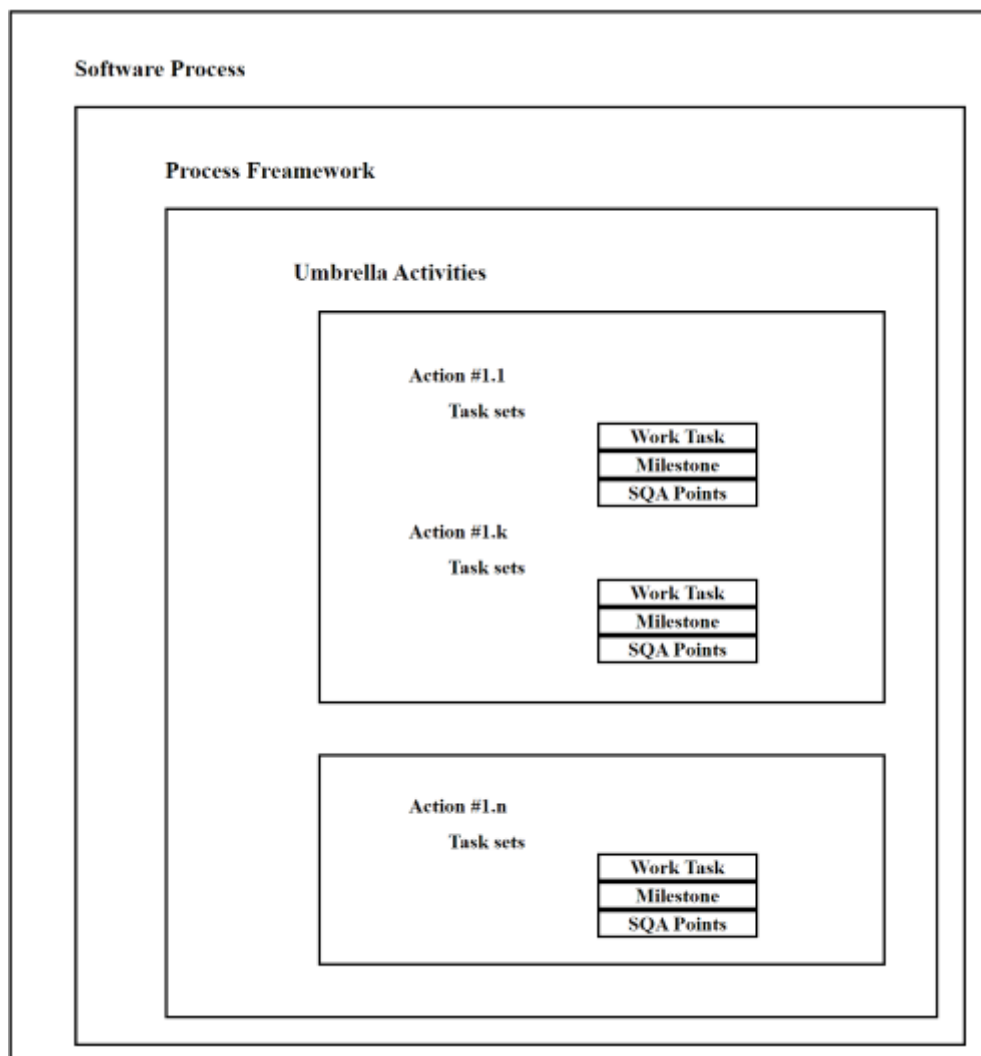
3. Method: During the process of software development the answers to all “how-to-do” questions are given by method. It has the information of all the tasks which includes communication, requirement analysis, design modeling, program construction, testing, and support.

4. Tools: Software engineering tools provide a self-operating system for processes and methods. Tools are integrated which means information created by one tool can be used by another.

Software Process Framework

Software Process Framework details the steps and chronological order of a process. Since it serves as a foundation for them, it is utilized in most applications. Task sets, umbrella activities, and process framework activities all define the characteristics of the software development process. Software Process includes:

1. **Tasks:** They focus on a small, specific objective.
2. **Action:** It is a set of tasks that produce a major work product.
3. **Activities:** Activities are groups of related tasks and actions for a major objective.



Software Process Framework

Process Framework Activities (C P M C D)

The process framework is required for representing common process activities. Five framework activities are described in a process framework for software engineering. Communication, planning, modeling, construction, and deployment are all examples of framework activities. Each engineering action defined by a framework activity comprises a list of needed work outputs, project milestones, and software quality assurance (SQA) points.

The following framework activities are carried out irrespective of the process model chosen by the organization.

1. Communication
2. Planning
3. Modeling
4. Construction
5. Deployment

The name 'prescriptive' is given because the model prescribes a set of activities, actions, tasks, quality assurance and change the mechanism for every project.

1. Communication

By communication, customer requirement gathering is done. Communication with consumers and stakeholders to determine the system's objectives and the software's requirements.

2. Planning

Establish engineering work plan, describes technical risk, lists resources requirements, work produced and defines work schedule.

3. Modeling

Architectural models and design to better understand the problem and to work towards the best solution. The software model is prepared by:

- Analysis of requirements
- Design

4. Construction

Creating code, testing the system, fixing bugs, and confirming that all criteria are met. The software design is mapped into a code by:

- Code generation
- Testing

5. Deployment

In this activity, a complete or non-complete product or software is represented to the customers to evaluate and give feedback. On the basis of their feedback, we modify the product for the supply of better products.

Umbrella Activities

Umbrella Activities are that take place during a software development process for improved project management and tracking.

1. **Software project tracking and control:** This is an activity in which the team can assess progress and take corrective action to maintain the schedule. Take action to keep the project on time by comparing the project's progress against the plan.
2. **Risk management:** The risks that may affect project outcomes or quality can be analyzed. Analyze potential risks that may have an impact on the software product's quality and outcome.
3. **Software quality assurance:** These are activities required to maintain software quality. Perform actions to ensure the product's quality.
4. **Formal technical reviews:** It is required to assess engineering work products to uncover and remove errors before they propagate to the next activity. At each level of the process, errors are evaluated and fixed.
5. **Software configuration management:** Managing of configuration process when any change in the software occurs.
6. **Work product preparation and production:** The activities to create models, documents, logs, forms, and lists are carried out.
7. **Reusability management:** It defines criteria for work product reuse. Reusable work items should be backed up, and reusable software components should be achieved.
8. **Measurement:** In this activity, the process can be defined and collected. Also, project and product measures are used to assist the software team in delivering the required software.

Imp

Process Model

- A software process model is a simplified representation of a software process, presented from a specific perspective.
- A software process model is an abstract representation of a software process.
- Attempt to organize the software life cycle by
 - defining activities involved in software production
 - order of activities and their relationships
- Every process model must be adapted so that it is used effectively for a specific software project.

Process models:

- ✓ Prescriptive Process Models
- ✓ Specialized Process Models.

Prescriptive Process Models

- Prescriptive software process models have been applied for many years in an effort to bring order and structure to software development.
- Each of these conventional models suggests a somewhat different process flow, but all perform the same set of *generic framework activities* –
 - Communication
 - Planning
 - Modeling
 - Construction
 - deployment

The name '**prescriptive**' is given because the model prescribes a set of activities, actions, tasks, quality assurance and change the mechanism for every project.

Prescriptive Process Models

- A prescriptive process model populates a process framework with explicit task sets for software engineering actions.
- Prescriptive process models advocate an orderly approach to software engineering.
- Prescriptive Process Models are NOT static, they should be adapted to the –

- People
- Problem
- Project

Perspective Process Model:

- A prescriptive model prescribes how a new software system should be developed.
- Prescriptive models are used as guidelines or frameworks to organize and structure how software development activities should be performed, and in what order.
- Following are the different Model:
 - *The Waterfall Model*
 - *Incremental Process Model*
 - *Evolutionary Process Model*
 - *Concurrent Model*

There are three types of prescriptive process models. They are:

1. The Waterfall Model
2. Incremental Process model
3. RAD model

1. The Waterfall Model

- The waterfall model is also called as '**Linear sequential model**' or '**Classic life cycle model**'.
- In this model, each phase is fully completed before the beginning of the next phase.
- This model is used for the small projects.
- In this model, feedback is taken after each phase to ensure that the project is on the right path.
- Testing part starts only after the development is complete.

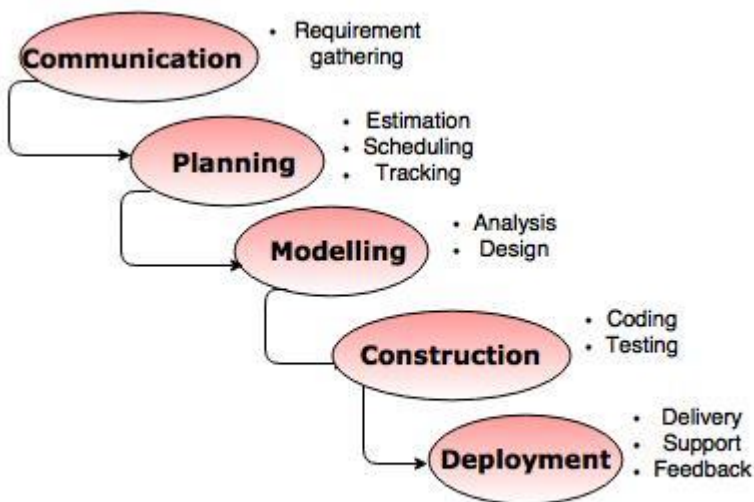


Fig. - The Waterfall model

NOTE: The description of the phases of the waterfall model is same as that of the process model.

An alternative design for 'linear sequential model' is as follows:

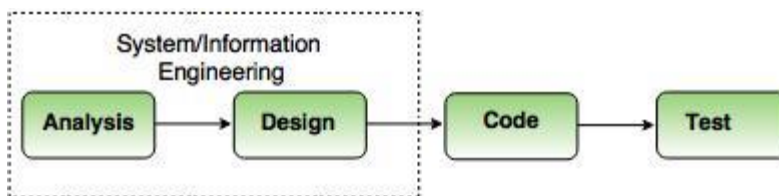
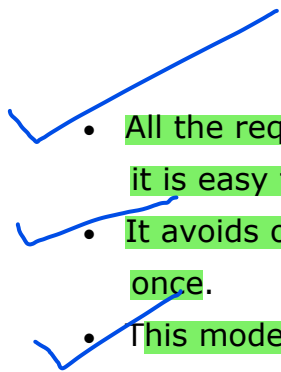


Fig. - The linear sequential model

Advantages of waterfall model

- The waterfall model is simple and easy to understand, implement, and use.

- 
- All the requirements are known at the beginning of the project, hence it is easy to manage.
 - It avoids overlapping of phases because each phase is completed at once.
 - This model works for small projects because the requirements are understood very well.
 - This model is preferred for those projects where the quality is more important as compared to the cost of the project.

Disadvantages of the waterfall model

- This model is not good for complex and object oriented projects.
- It is a poor model for long projects.
- The problems with this model are uncovered, until the software testing.
- The amount of risk is high.

2. Incremental Process model

- The incremental model combines the elements of waterfall model and they are applied in an iterative fashion.
- The first increment in this model is generally a core product.
- Each increment builds the product and submits it to the customer for any suggested modifications.
- The next increment implements on the customer's suggestions and add additional requirements in the previous increment.
- This process is repeated until the product is finished.

For example, the word-processing software is developed using the incremental model.

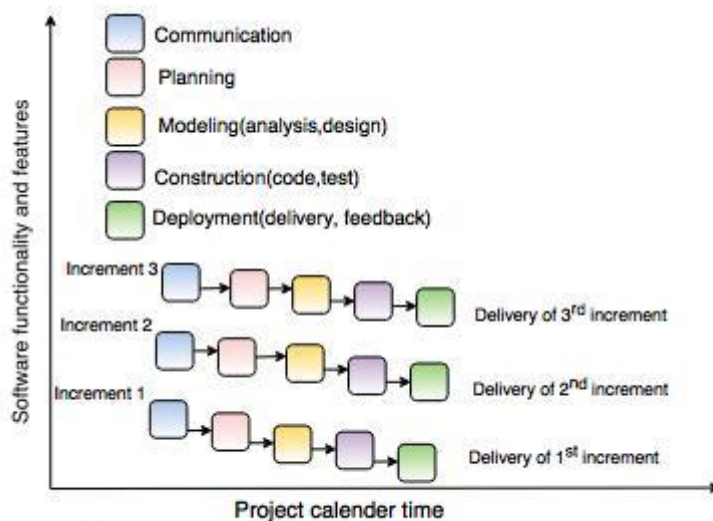


Fig. - Incremental Process Model

Advantages of incremental model

- This model is flexible because the cost of development is low and initial product delivery is faster.
- ✓ • It is easier to test and debug during the smaller iteration.
- ✓ • The working software generates quickly and early during the software life cycle.
- ✓ • The customers can respond to its functionalities after every increment.

Disadvantages of the incremental model

- ✓ • The cost of the final product may cross the cost estimated initially.
- This model requires a very clear and complete planning.
- The planning of design is required before the whole system is broken into small increments.
- The demands of customer for the additional functionalities after every increment causes problem during the system architecture.

3. RAD model

- RAD is a Rapid Application Development model.
- Using the RAD model, software product is developed in a short period of time.
- The initial activity starts with the communication between customer and developer.
- Planning depends upon the initial requirements and then the requirements are divided into groups.

- Planning is more important to work together on different modules.

The RAD model consist of following phases:

1. Business Modeling

- Business modeling consist of the flow of information between various functions in the project.
- For example what type of information is produced by every function and which are the functions to handle that information.
- A complete business analysis should be performed to get the essential business information.

2. Data modeling

- The information in the business modeling phase is refined into the set of objects and it is essential for the business.
- The attributes of each object are identified and define the relationship between objects.

3. Process modeling

- The data objects defined in the data modeling phase are changed to fulfil the information flow to implement the business model.
- The process description is created for adding, modifying, deleting or retrieving a data object.

4. Application generation

- In the application generation phase, the actual system is built.
- To construct the software the automated tools are used.

5. Testing and turnover

- The prototypes are independently tested after each iteration so that the overall testing time is reduced.
- The data flow and the interfaces between all the components are fully tested. Hence, most of the programming components are already tested.

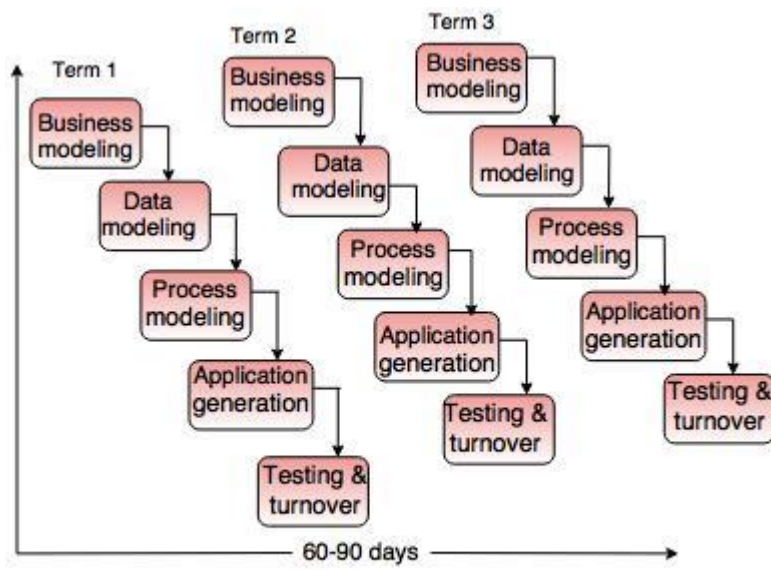


Fig. - RAD Model

Specialised Process Models

Specialized process models take on many of the characteristics of one or more of the traditional models. However, these models tend to be applied when a specialized or narrowly defined software engineering approach is chosen.

There are 3 types of specialized process models:

1. Component Based Development
2. Formal Methods Model
3. Aspect Oriented Software development

1. Component Based Development :

Commercial off-the-shelf (COTS) software components, developed by vendors who offer them as products, provide targeted functionality with well-defined interfaces that enable the component to be integrated into the software that is to be built.

Regardless of the technology that is used to create the components, the **component-based development model incorporates the following steps:**

1. Available component-based products are researched and evaluated for the application domain in question.
2. Component integration issues are considered.
3. A software architecture is designed to accommodate the components.
4. Components are integrated into the architecture.
5. Comprehensive testing is conducted to ensure proper functionality

The component-based development model leads to software reuse, and reusability provides software engineers with a number of measurable benefits.

Software engineering team can achieve a reduction in development cycle time as well as a reduction in project cost if component reuse becomes part of your culture.

2. **Formal Methods Model :**

The formal methods model encompasses a set of activities that leads to formal mathematical specification of computer software.

- Formal methods enable to specify, develop, and verify a computer-based system by applying a rigorous, mathematical notation.
- When formal methods are used during development, they provide a mechanism for eliminating many of the problems that are difficult to overcome using other software engineering paradigms.
- Ambiguity, incompleteness, and inconsistency can be discovered and corrected more easily, through the application of mathematical analysis.
- When formal methods are used during design, they serve as a basis for program verification and therefore enable you to discover and correct errors that might otherwise go undetected.

3. **Aspect Oriented Software Development :**

Software developers of complex software invariably implement a set of localized features, functions, and information content.

These localized software characteristics are modeled as components and then constructed within the context of a system architecture.

Aspect-oriented software development (AOSD), often referred to as aspect-oriented programming (AOP), is a relatively new software engineering paradigm that provides a process and methodological approach for defining, specifying, designing, and constructing aspects.

It is essential to instantiate asynchronous communication between the software process activities applied to the engineering and construction of aspects and components.

The evolutionary model is appropriate as aspects are identified and then constructed.

Agile Software Development

Agile Software Development is a [software development methodology](#) that values flexibility, collaboration, and customer satisfaction.

Agile Software Development is an iterative and incremental approach to [software development](#) that emphasizes the importance of delivering a working product quickly and frequently.

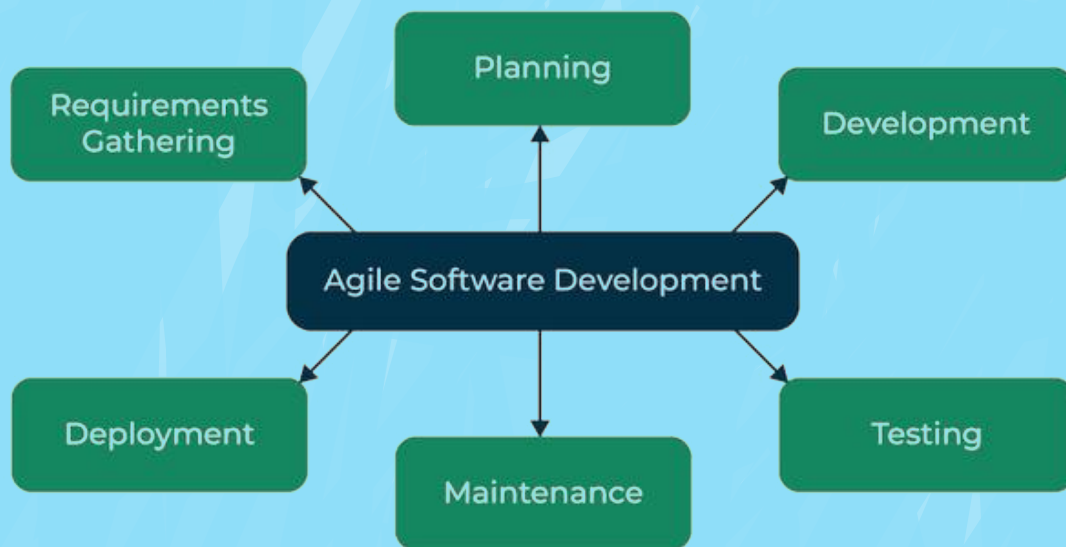
Agile software development is an iterative and incremental approach to software development.

It emphasizes collaboration between the development team and the customer, flexibility, and adaptability in the face of changing requirements, and the delivery of working software in short iterations.

Principles of Agile:

1. The highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. It welcomes changing requirements, even late in development.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for the shortest timescale.
4. Build projects around motivated individuals. Give them the environment and the support they need and trust them to get the job done.
5. Working software is the primary measure of progress.
6. Simplicity the art of maximizing the amount of work not done is essential.
7. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
8. By the amount of work that has been finished, gauge your progress.
9. Never give up on excellence.
10. Take advantage of change to gain a competitive edge.

The Agile Software Development Process:



Agile Software Development

1. **Requirements Gathering:** The customer's requirements for the software are gathered and prioritized.
2. **Planning:** The development team creates a plan for delivering the software, including the features that will be delivered in each iteration.
3. **Development:** The development team works to build the software, using frequent and rapid iterations.
4. **Testing:** The software is thoroughly tested to ensure that it meets the customer's requirements and is of high quality.
5. **Deployment:** The software is deployed and put into use.
6. **Maintenance:** The software is maintained to ensure that it continues to meet the customer's needs and expectations.

Agile Software Development is widely used by software development teams and is considered to be a flexible and adaptable approach to software development that is well-suited to changing requirements and the fast pace of software development.

Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver all at once.

Agile Software Development Cycle



Agile software development cycle

Advantages of Agile Software Development over traditional software development approaches:

1. **Increased customer satisfaction:** Agile development involves close collaboration with the customer, which helps to ensure that the software meets their needs and expectations.
2. **Faster time-to-market:** Agile development emphasizes the delivery of working software in short iterations, which helps to get the software to market faster.
3. **Reduced risk:** Agile development involves frequent testing and feedback, which helps to identify and resolve issues early in the development process.
4. **Improved team collaboration:** Agile development emphasizes collaboration and communication between team members, which helps to improve productivity and morale.
5. **Adaptability to change:** Agile Development is designed to be flexible and adaptable, which means that changes to the project scope, requirements, and timeline can be accommodated easily. This can help the team to respond quickly to changing business needs and market demands.
6. **Better quality software:** Agile Development emphasizes continuous testing and feedback, which helps to identify and resolve issues early in the development process. This can lead to higher-quality software that is more reliable and less prone to errors.

7. **Increased transparency:** Agile Development involves frequent communication and collaboration between the team and the customer, which helps to improve transparency and visibility into the project status and progress. This can help to build trust and confidence with the customer and other stakeholders.
8. **Higher productivity:** Agile Development emphasizes teamwork and collaboration, which helps to improve productivity and reduce waste. This can lead to faster delivery of working software with fewer defects and rework.
9. **Improved project control:** Agile Development emphasizes continuous monitoring and measurement of project metrics, which helps to improve project control and decision-making. This can help the team to stay on track and make data-driven decisions throughout the development process.

Disadvantages Agile Software Development:

- In the case of large software projects, **it is difficult to assess the effort required at the initial stages** of the software development life cycle.
- Agile Development is more code-focused and produces **less documentation**.
- **Agile development is heavily dependent on the inputs of the customer.** If the customer has ambiguity in his vision of the outcome, it is highly likely that the project to get off track.
- **Face-to-face communication is harder** in large-scale organizations.
- **Only senior programmers are capable of making the kind of decisions** required during the development process. Hence, it's a difficult situation for new programmers to adapt to the environment.
- **Lack of predictability:** Agile Development relies heavily on customer feedback and continuous iteration, which can make it difficult to predict project outcomes, timelines, and budgets.
- **Limited scope control:** Agile Development is designed to be flexible and adaptable, which means that scope changes can be easily accommodated. However, this can also lead to scope creep and a lack of control over the project scope.
- **Lack of emphasis on testing:** Agile Development places a greater emphasis on delivering working code quickly, which can lead to a

lack of focus on testing and quality assurance. This can result in bugs and other issues that may go undetected until later stages of the project.

- **Risk of team burnout:** Agile Development can be intense and fast-paced, with frequent sprints and deadlines. This can put a lot of pressure on team members and lead to burnout, especially if the team is not given adequate time for rest and recovery.
- **Lack of structure and governance:** Agile Development is often less formal and structured than other development methodologies, which can lead to a lack of governance and oversight. This can result in inconsistent processes and practices, which can impact project quality and outcomes.

What is Extreme Programming (XP)?

Extreme programming (XP) is one of the most important software development frameworks of Agile models. It is used to improve software quality and responsiveness to customer requirements. The extreme programming model recommends taking the best practices that have worked well in the past in program development projects to extreme levels.

Good practices need to be practiced in extreme programming: Some of the good practices that have been recognized in the extreme programming model and suggested to maximize their use are given below:

- **Code Review:** Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their work between them every hour.
- **Testing:** Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.
- **Incremental development:** Incremental development is very good because customer feedback is gained and based on this development team comes up with new increments every few days after each iteration.
- **Simplicity:** Simplicity makes it easier to develop good-quality code as well as to test and debug it.
- **Design:** Good quality design is important to develop good quality software. So, everybody should design daily.
- **Integration testing:** It helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

Basic principles of Extreme programming:

XP is based on the frequent iteration through which the developers implement User Stories.

A User Story is a conventional description by the user of a feature of the required system. It does not mention finer details such as the different scenarios that can occur.

Based on User stories, the project team proposes Metaphors. Metaphors are a common vision of how the system would work.

The development team may decide to build a Spike for some features.

A Spike is a very simple program that is constructed to explore the suitability of a solution being proposed. It can be considered similar to a prototype.

Some of the **basic activities** that are followed during software development by using the XP model are given below:

- **Coding:** The concept of coding which is used in the XP model is slightly different from traditional coding. Here, the coding activity includes drawing diagrams (modeling) that will be transformed into code, scripting a web-based system, and choosing among several alternative solutions.
- **Testing:** The XP model gives high importance to testing and considers it to be the primary factor in developing fault-free software.
- **Listening:** The developers need to carefully listen to the customers if they have to develop good quality software. Sometimes programmers may not have the depth knowledge of the system to be developed. So, the programmers should understand properly the functionality of the system and they have to listen to the customers.
- **Designing:** Without a proper design, a system implementation becomes too complex, and very difficult to understand the solution, thus making maintenance expensive. A good design results elimination of complex dependencies within a system. So, effective use of suitable design is emphasized.
- **Feedback:** One of the most important aspects of the XP model is to gain feedback to understand the exact customer needs. Frequent contact with the customer makes the development effective.
- **Simplicity:** The main principle of the XP model is to develop a simple system that will work efficiently in the present time, rather than trying to build something that would take time and may never be used. It focuses on some specific features that are immediately needed, rather than engaging time and effort on speculations of future requirements.
- **Pair Programming:** XP encourages pair programming where two developers work together at the same workstation. This approach helps in knowledge sharing, reduces errors, and improves code quality.
- **Continuous Integration:** In XP, developers integrate their code into a shared repository several times a day. This helps to detect and resolve integration issues early on in the development process.

- **Refactoring:** XP encourages refactoring, which is the process of restructuring existing code to make it more efficient and maintainable. Refactoring helps to keep the codebase clean, organized, and easy to understand.
- **Collective Code Ownership:** In XP, there is no individual ownership of code. Instead, the entire team is responsible for the codebase. This approach ensures that all team members have a sense of ownership and responsibility towards the code.
- **Planning Game:** XP follows a planning game, where the customer and the development team collaborate to prioritize and plan development tasks. This approach helps to ensure that the team is working on the most important features and delivers value to the customer.
- **On-site Customer:** XP requires an on-site customer who works closely with the development team throughout the project. This approach helps to ensure that the customer's needs are understood and met, and also facilitates communication and feedback.

Applications of Extreme Programming (XP): Some of the projects that are suitable to develop using the XP model are given below:

- **Small projects:** The XP model is very useful in small projects consisting of small teams as face-to-face meeting is easier to achieve.
- **Projects involving new technology or Research projects:** This type of project faces changing requirements rapidly and technical problems. So XP model is used to complete this type of project.
- **Web development projects:** The XP model is well-suited for web development projects as the development process is iterative and requires frequent testing to ensure the system meets the requirements.
- **Collaborative projects:** The XP model is useful for collaborative projects that require close collaboration between the development team and the customer.
- **Projects with tight deadlines:** The XP model can be used in projects that have a tight deadline, as it emphasizes simplicity and iterative development.
- **Projects with rapidly changing requirements:** The XP model is designed to handle rapidly changing requirements, making it suitable for projects where requirements may change frequently.

- **Projects where quality is a high priority:** The bureaucracy-based XP model places a strong emphasis on testing and quality assurance, making it a suitable approach for projects where quality is a high priority.

Extreme Programming (XP) is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation.

XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment.

XP includes the following practices:

1. Continuous Integration: Code is integrated and tested frequently, with all changes reviewed by the development team.
2. Test-Driven Development: Tests are written before code is written, and the code is developed to pass those tests.
3. Pair Programming: Developers work together in pairs to write code and review each other's work.
4. Continuous Feedback: Feedback is obtained from customers and stakeholders through frequent demonstrations of working software.
5. Simplicity: XP prioritizes simplicity in design and implementation, to reduce complexity and improve maintainability.
6. Collective Ownership: All team members are responsible for the code, and anyone can make changes to any part of the codebase.
7. Coding Standards: Coding standards are established and followed to ensure consistency and maintainability of the code.
8. Sustainable Pace: The pace of work is maintained at a sustainable level, with regular breaks and opportunities for rest and rejuvenation.
9. XP is well-suited to projects with rapidly changing requirements, as it emphasizes flexibility and adaptability. It is also well-suited to projects with tight timelines, as it emphasizes rapid development and deployment.
10. Refactoring: Code is regularly refactored to improve its design and maintainability, without changing its functionality.
11. Small Releases: Software is released in small increments, allowing for frequent feedback and adjustments based on that feedback.

12. Customer Involvement: Customers are actively involved in the development process, providing feedback and clarifying requirements.
13. On-Site Customer: A representative from the customer's organization is present with the development team to provide continuous feedback and answer questions.
14. Short Iterations: Work is broken down into short iterations, usually one to two weeks in length, to allow for rapid development and frequent feedback.
15. Planning Game: The team and customer work together to plan and prioritize the work for each iteration, to deliver the most valuable features first.
16. Metaphor: A shared metaphor is used to guide the design and implementation of the system.
17. Coding Standards: Coding standards are established and followed to ensure consistency and maintainability of the code.

Advantages of Extreme Programming (XP):

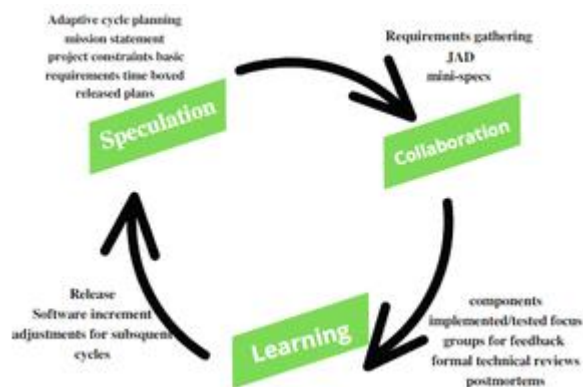
- **Slipped schedules** – Timely delivery is ensured through slipping timetables and doable development cycles.
- **Misunderstanding the business and/or domain** – Constant contact and explanations are ensured by including the client on the team.
- **Cancelled projects** – Focusing on ongoing customer engagement guarantees open communication with the consumer and prompt problem-solving.
- **Staff turnover** – Teamwork that is focused on cooperation provides excitement and goodwill. Team spirit is fostered by multidisciplinary cohesion.
- **Costs incurred in changes** – Extensive and continuing testing ensures that the modifications do not impair the functioning of the system. A functioning system always guarantees that there is enough time to accommodate changes without impairing ongoing operations.
- **Business changes** – Changes are accepted at any moment since they are seen to be inevitable.
- **Production and post-delivery defects: Emphasis is on** – the unit tests to find and repair bugs as soon as possible.

Adaptive Software Development (ASD)

Adaptive Software Development is a method to build complex software and system. ASD focuses on human collaboration and self-organization. Adaptive software development (ASD) is a [software development](#) process that grew out of the work by Jim Highsmith and Sam Bayer on rapid application development (RAD). ASD “**life cycle**” incorporates three phases namely:

1. Speculation
2. Collaboration
3. Learning

These are explained as following below.



Adaptive Software Development

1. Speculation:

During this phase project is initiated and planning is conducted. The project plan uses project initiation information like project requirements, user needs, customer mission statement, etc, to define set of release cycles that the project wants.

2. Collaboration:

It is the difficult part of ASD as it needs the workers to be motivated. It collaborates communication and teamwork but emphasizes individualism as individual creativity plays a major role in creative thinking. People working together must trust each others to

- Criticize without animosity,
- Assist without resentment,

- Work as hard as possible,
- Possession of skill set,
- Communicate problems to find effective solution.

3. Learning:

The workers may have a overestimate of their own understanding of the technology which may not lead to the desired result. Learning helps the workers to increase their level of understanding over the project.

Learning process is of 3 ways:

1. Focus groups
2. Technical reviews
3. Project postmortem

ASD's overall emphasis on the dynamics of self-organizing teams, interpersonal collaboration, and individual and team learning yield software project teams that have a much higher likelihood of success.

Scrum (software development)

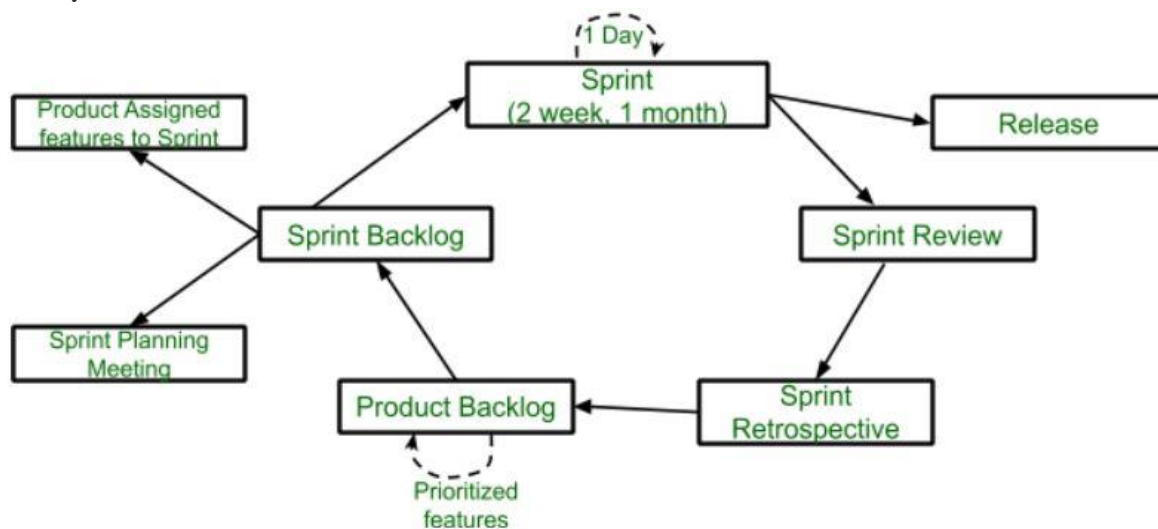
Scrum is the type of **Agile framework**. It is a framework within which people can address complex adaptive problem while productivity and creativity of delivering product is at highest possible values. **Scrum uses Iterative process. Silent features of Scrum are:**

- Scrum is light-weighted framework
- Scrum emphasizes self-organization
- Scrum is simple to understand
- Scrum framework help the team to work together

Lifecycle

of

Scrum:



1. **Sprint:** A Sprint is a time box of one month or less. A new Sprint starts immediately after the completion of the previous Sprint.
2. **Release:** When the product is completed, it goes to the Release stage.
3. **Sprint Review:** If the product still has some non-achievable features, it will be checked in this stage and then passed to the Sprint Retrospective stage.
4. **Sprint Retrospective:** In this stage quality or status of the product is checked.
5. **Product Backlog:** According to the prioritize features the product is organized.
6. **Sprint Backlog:** Sprint Backlog is divided into two parts Product assigned features to sprint and Sprint planning meeting.

Advantage of using Scrum framework:

- Scrum framework is fast moving and money efficient.
- Scrum framework works by dividing the large product into small sub-products. It's like a divide and conquer strategy
- In Scrum customer satisfaction is very important.
- Scrum is adaptive in nature because it have short sprint.
- As Scrum framework rely on constant feedback therefore the quality of product increases in less amount of time

Disadvantage of using Scrum framework:

- Scrum framework do not allow changes into their sprint.
- Scrum framework is not fully described model. If you wanna adopt it you need to fill in the framework with your own details like Extreme Programming(XP), Kanban, DSDM.
- It can be difficult for the Scrum to plan, structure and organize a project that lacks a clear definition.
- The daily Scrum meetings and frequent reviews require substantial resources.

Dynamic Systems Development Method (DSDM)

The **Dynamic Systems Development technique (DSDM)** is an associate degree agile code development approach that provides a framework for building and maintaining systems.

The DSDM philosophy is borrowed from a modified version of the sociologist principle—80 % of An application is often delivered in 20% of the time it'd desire deliver the entire (100 percent) application.

DSDM is An iterative code method within which every iteration follows the 80% rule that simply enough work is needed for every increment to facilitate movement to the following increment. The remaining detail is often completed later once a lot of business necessities are noted or changes are requested and accommodated.

The DSDM tool (www.dsdm.org) could be a worldwide cluster of member companies that put together tackle the role of “keeper” of the strategy. The pool has outlined AN [Agile Development Model](#), known as the DSDM life cycle that defines 3 different unvarying cycles, preceded by 2 further life cycle activities

life cycle activities:

1. **Feasibility Study:**

It establishes the essential business necessities and constraints related to the applying to be designed then assesses whether or not the application could be a viable candidate for the DSDM method.

2. **Business Study:**

It establishes the use and knowledge necessities that may permit the applying to supply business value; additionally, it is the essential application design and identifies the maintainability necessities for the applying.

Different Unvarying Cycles

1. **Functional Model Iteration:**

It produces a collection of progressive prototypes that demonstrate practicality for the client. (Note: All DSDM prototypes are supposed to evolve into the deliverable application.) The intent throughout this unvarying cycle is to collect further necessities by eliciting feedback from users as they exercise the paradigm.

2. **Design and Build Iteration:**

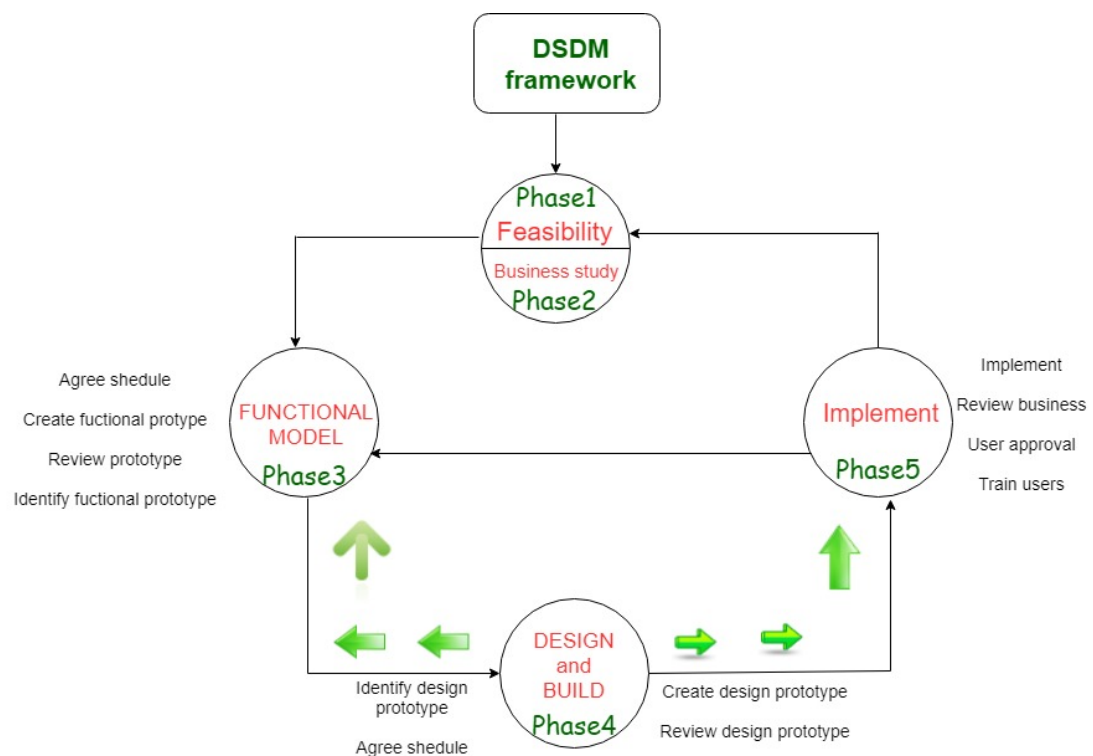
It revisits prototypes designed throughout useful model iteration to make sure that everyone has been designed during a manner that may alter it to supply operational business price for finish users. In some cases, useful model iteration and style and build iteration occur at the same time.

3. **Implementation:**

It places the newest code increment (an “operationalized” prototype) into the operational surroundings. It ought to be noted that:

- **(a)** the increment might not 100% complete or,
- **(b)** changes are also requested because the increment is placed into place. In either case, DSDM development work continues by returning to the useful model iteration activity.

Below diagram describe the DSDM life cycle:



Dynamic Systems Development Method life cycle

DSDM is often combined with XP to supply a mixed approach that defines a solid method model (the DSDM life cycle) with the barmy and bolt practices (XP) that are needed to create code increments. additionally, the ASD ideas of collaboration and self-organizing groups are often tailored to a combined method model.

Crystal methods in Agile Development/Framework

Crystal methods in Agile Development/Framework: The crystal method is an agile framework that is considered a lightweight or agile methodology that focuses on individuals and their interactions. The

methods are color-coded to significant risk to human life. It is mainly for short-term projects by a team of developers working out of a single workspace.

Two core beliefs of the Crystal method:

- Find your own way and methods to optimize workflow.
- Make use of unique methods to make the project unique and dynamic.

Properties of Crystal Agile Framework:

1. **Frequent Delivery-** It allows you regularly deliver the products and test code to real users. Without this, you might build a product that nobody needs.
2. **Reflective Improvement-** No matter how good you have done or how bad you have done. Since there are always areas where the product can be improved, so the teams can implement to improve their future practices.
3. **Osmotic Communication-** Alistair stated that having the teams in the same physical phase is very much important as it allows information to flow in between members of a team as in osmosis.
4. **Personal Safety-** There are no bad suggestions in a crystal team, team members should feel safe to discuss ideas openly without any fear.
5. **Focus-** Each member of the team knows exactly what to do, which enables them to focus their attention. This boosts team interaction and works towards the same goal.
6. **Easy access to expert users-** It enhances team communication with users and gets regular feedback from real users.
7. **Technical tooling-** It contains very specific technical tools which to be used by the software development team during testing, management, and configuration. These tools make it enable the team to identify any error within less time.

8. **Continuous learning** – The framework emphasizes on continuous learning, enabling team members to acquire new skills and knowledge, and apply them in their work.
9. **Teamwork** – The framework stresses on the importance of teamwork, promoting collaboration, and mutual support among team members.
10. **Timeboxing** – The framework adopts timeboxing to manage project deadlines, ensuring that the team delivers within set timelines.
11. **Incremental development** – The framework promotes incremental development, enabling the team to deliver working software frequently, and adapt to changes as they arise.
12. **Automated testing** – The framework emphasizes on automated testing, enabling the team to detect and fix bugs early, reducing the cost of fixing errors at later stages.
13. **Customer involvement** – The framework emphasizes on involving customers in the development process, promoting customer satisfaction, and delivering products that meet their needs.
14. **Leadership** – The framework encourages leadership, enabling team members to take ownership of their work and make decisions that contribute to the success of the project.



CRYSTAL FAMILY (TEAM MEMBERS)

Benefits of using the Crystal Agile Framework :

- Facilitate and enhance team communication and accountability.
- The adaptive approach lets the team respond well to the demanding requirements.
- Allows team to work with the one they see as the most effective.
- Teams talk directly with each other, which reduces management overhead.
- **Faster delivery** – The framework enables the team to deliver working software faster, which can help gain a competitive advantage in the market.
- **Higher quality** – The framework emphasizes on quality, enabling the team to detect and fix defects early in the development process, resulting in a higher quality product.
- **Improved customer satisfaction** – The framework promotes customer involvement, enabling the team to deliver products that meet customer needs, resulting in higher customer satisfaction.
- **Increased productivity** – The framework enables the team to focus on delivering the highest value features, which can increase productivity and reduce waste.
- **Flexibility** – The framework is highly adaptable, enabling the team to adjust to changing requirements, and make decisions based on real-time feedback.
- **Empowerment** – The framework promotes empowerment, enabling team members to take ownership of their work, and make decisions that contribute to the success of the project.
- **Reduced risk** – The framework promotes risk management, enabling the team to identify and mitigate potential risks early in the development process, reducing the likelihood of project failure.

Drawbacks of using the Crystal Agile Framework :

- A lack of pre-defined plans may lead to confusion and loss of focus.
- Lack of structure may slow down inexperienced teams.

- Not clear on how a remote team can share knowledge informally.
- **Lack of predictability** – The framework's emphasis on adaptability and flexibility may result in a lack of predictability, making it difficult to plan and estimate project timelines and budgets.
- **Lack of documentation** – The framework's emphasis on communication and collaboration may result in a lack of documentation, making it difficult to track progress and maintain a record of decisions.
- **Limited scalability** – The framework may not be suitable for large or complex projects, as the lack of structure and predefined plans may make it difficult to manage teams at scale.
- **Dependence on team expertise** – The framework relies heavily on the expertise and skills of the development team, which may not be suitable for teams with limited experience or knowledge.
- **Lack of clarity on roles and responsibilities** – The framework's emphasis on self-organizing teams may result in a lack of clarity on roles and responsibilities, leading to confusion and a loss of focus.
- **Inability to handle regulatory requirements** – The framework may not be suitable for projects with strict regulatory requirements, as the lack of documentation and structure may not meet compliance standards.
- **Potential for informal knowledge sharing** – The framework's emphasis on osmotic communication may result in informal knowledge sharing, which may be difficult to track and monitor for accuracy and completeness.

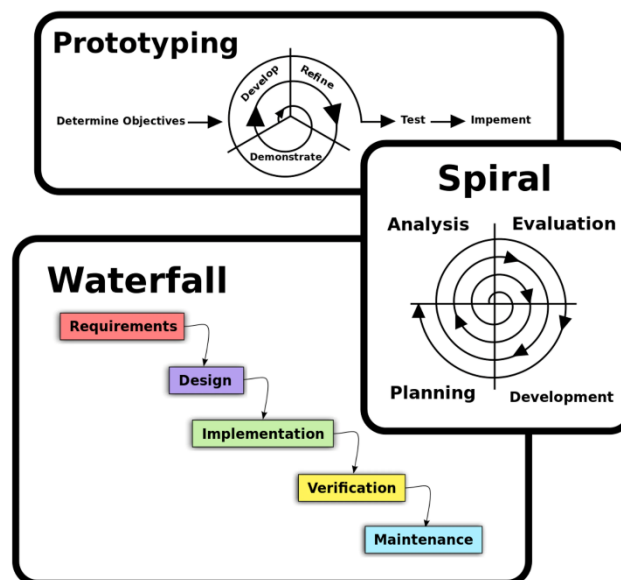
Selection Criteria for software process model

A software process model is an abstraction of the software development process. The models specify the stages and order of a process. So, think of this as a representation of the order of activities of the process and the sequence in which they are performed.

A model will define the following:

- The tasks to be performed
- The input and output of each task
- The pre and post-conditions for each task
- The flow and sequence of each task

The goal of a software process model is to provide guidance for controlling and coordinating the tasks to achieve the end product and objectives as effectively as possible.



There are many kinds of process models for meeting different requirements. We refer to these as SDLC models (Software Development Life Cycle models). The most popular and important SDLC models are as follows:

- Waterfall model
- Incremental model
- RAD model
- Spiral model

Factors in choosing a software process

Choosing the right software process model for your project can be difficult. If you know your requirements well, it will be easier to select a model that best matches your needs. You need to keep the following factors in mind when selecting your software process model:

Project requirements

Before you choose a model, take some time to go through the project requirements and clarify them alongside your organization's or team's expectations. Will the user need to specify requirements in detail after each iterative session? Will the requirements *change* during the development process?

Project size

Consider the size of the project you will be working on. Larger projects mean bigger teams, so you'll need more extensive and elaborate project management plans.

Project complexity

Complex projects may not have clear requirements. The requirements may change often, and the cost of delay is high. Ask yourself if the project requires constant monitoring or feedback from the client.

Cost of delay

Is the project highly time-bound with a huge cost of delay, or are the timelines flexible?

Customer involvement

Do you need to consult the customers during the process? Does the user need to participate in all phases?

Familiarity with technology

This involves the developers' knowledge and experience with the project domain, software tools, language, and methods needed for development.

Project resources

This involves the amount and availability of funds, staff, and other resources.

Parameter	Process Model→	Waterfall Model	Incremental Model	Prototype Model	Rad Model	Spiral Model	Agile Model	Xp programming
Clear Requirement Specifications		Initial level	Initial level	At medium level	Initial level	Initial level	Change incrementally	Initial level
Feedback from user		No	No	Yes	No	No	No	Yes
Speed to change		Low	High	Medium	No	High	High	High
Predictability		Low	Low	High	Low	Medium	High	High
Risk identification		At initial level	No	No	No	Yes	Yes	Yes
Practically implementation		No	Low	Medium	No	Medium	High	High
Loom		Systematic sequence	Iterative sequence	Priority on customer feedback	Use readymade component	Identification of risk at each stage	Highly customer satisfaction and incremental development[09]	Customer satisfaction and incremental development
Any variation done		Yes-v model	No	No	No	Yes-win win spiral[6]	No	No
Understandability		Simple	Intermediate	Intermediate	Intermediate	Hard	Much complex	Intermediate
Precondition		Requirement clearly defined	Core product should clearly define	Clear idea of Quick Design	Clean idea of Reuse component	No	No	No
Usability		Basic	Medium	High	Medium	Medium	Most use now a days	medium
Customer priority		Nil	Nil	Intermediate	Nil	Intermediate	High	Intermediate
Industry approach		Basic	Basic	Medium	Medium	Medium	High	Medium
Cost		Low	Low	High	very high	Expensive	Much Expensive	High
Resource organization		Yes	Yes	Yes	Yes	No	No	Yes
Elasticity		No	No	Yes	Yes	No	Very high	Medium

Types of software process models

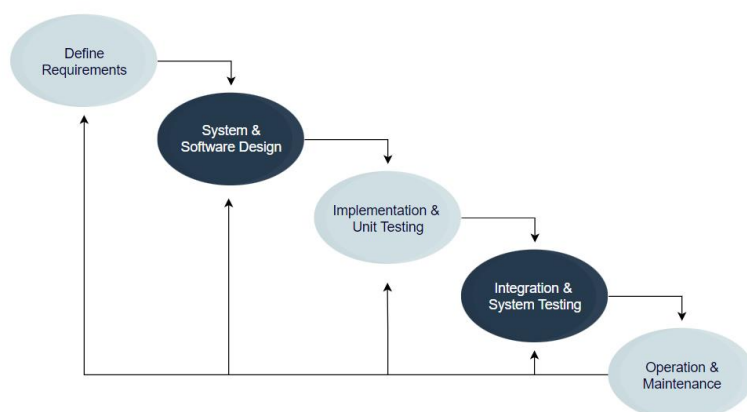
As we mentioned before, there are multiple kinds of software process models that each meet different requirements. Below, we will look at the top seven types of software process models that you should know.

Waterfall Model

The waterfall model is a **sequential, plan driven-process** where you must plan and schedule all your activities before starting the project. Each activity in the waterfall model is represented as a separate phase arranged in linear order.

It has the following phases:

- Requirements
- Design
- Implementation
- Testing
- Deployment
- Maintenance



Each of these phases produces one or more documents that need to be approved before the next phase begins. However, in practice, these

phases are very likely to overlap and may feed information to one another.

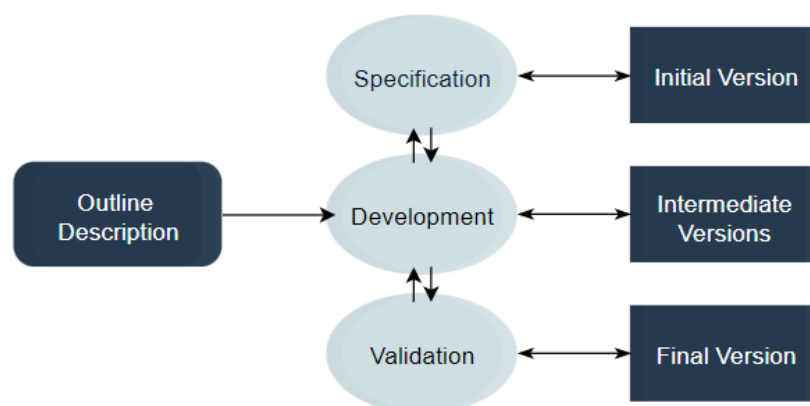
The software process **isn't linear**, so the documents produced may need to be modified to reflect changes.

The waterfall model is easy to understand and follow. It doesn't require a lot of customer involvement after the specification is done. Since it's inflexible, it can't adapt to changes. There is no way to see or try the software until the last phase.

The waterfall model has a rigid structure, so it should be used in cases where the requirements are understood completely and unlikely to radically change.

Incremental Model

The incremental model divides the system's functionality into **small increments** that are delivered one after the other in quick succession. The most important functionality is implemented in the initial increments.



The subsequent increments expand on the previous ones until everything has been updated and implemented.

Incremental development is based on developing an initial implementation, exposing it to user feedback, and evolving it through new versions. The process' activities are interwoven by feedback.

Each iteration passes through the requirements, design, coding, and testing stages.

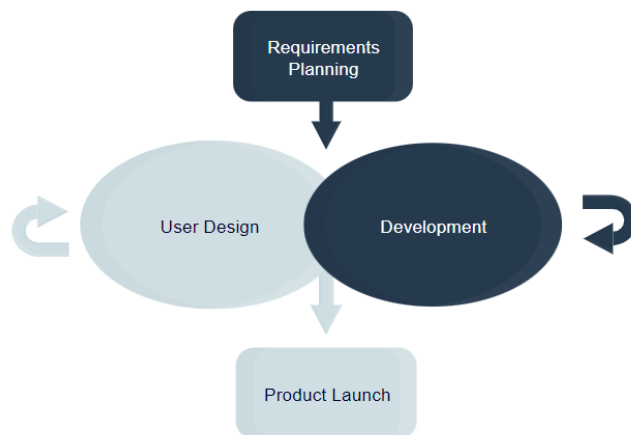
The incremental model lets stakeholders and developers see results with the first increment. If the stakeholders don't like anything, everyone finds out a lot sooner. It is efficient as the developers only focus on what is important and bugs are fixed as they arise, but you need a **clear and complete definition** of the whole system before you start.

The incremental model is great for projects that have loosely coupled parts and projects with complete and clear requirements.

RAD Model

The Rapid Application Development (RAD model) is based on iterative development and prototyping with **little planning involved**. You develop functional modules in parallel for faster product delivery. It involves the following phases:

1. Business modeling
2. Data modeling
3. Process modeling
4. Application generation
5. Testing and turnover



The RAD concept focuses on gathering requirements using focus groups and workshops, reusing software components, and informal communication.

The RAD model accommodates changing requirements, reduces development time, and increases the reusability of components. But it can be complex to manage. Therefore, the RAD model is great for systems that need to be produced in a short time and have known requirements.

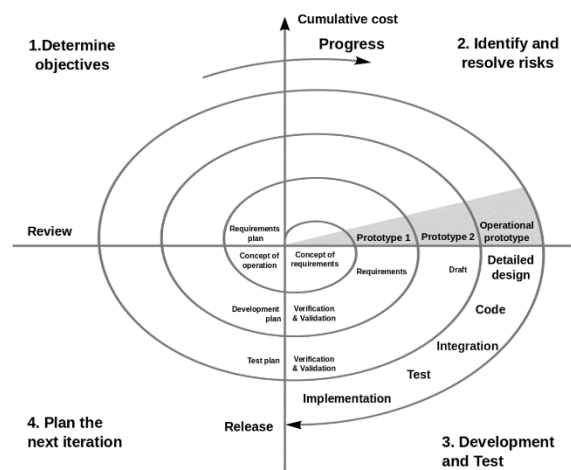
Spiral Model

The spiral model is a risk driven iterative software process model. The spiral model delivers projects in loops. Unlike other process models, its steps aren't activities but **phases** for addressing whatever problem has the greatest risk of causing a failure.

It was designed to include the best features from the waterfall and introduces risk-assessment.

You have the following phases for each cycle:

1. Address the highest-risk problem and determine the objective and alternate solutions
2. Evaluate the alternatives and identify the risks involved and possible solutions
3. Develop a solution and verify if it's acceptable
4. Plan for the next cycle



You develop the concept in the first few cycles, and then it evolves into an implementation. Though this model is great for managing uncertainty, it can be difficult to have stable documentation. The spiral model can be used for projects with **unclear needs** or projects still in research and development.

Selection Process Parameters for a Software Life Cycle Model

Selection Process parameters play an important role in software development as it helps to choose the best suitable software life cycle model. Following are the parameters which should be used to select a SDLC.

1. Requirements characteristics :

- Reliability of Requirements
- How often the requirements can change
- Types of requirements
- Number of requirements
- Can the requirements be defined at an early stage
- Requirements indicate the complexity of the system

2. Development team :

- Team size
- Experience of developers on similar type of projects
- Level of understanding of user requirements by the developers
- Environment
- Domain knowledge of developers
- Experience on technologies to be used
- Availability of training

3. User involvement in the project :

- Expertise of user in project
- Involvement of user in all phases of the project
- Experience of user in similar project in the past

4. Project type and associated risk :

- Stability of funds
- Tightness of project schedule
- Availability of resources
- Type of project
- Size of the project
- Expected duration for the completion of project
- Complexity of the project
- Level and the type of associated risk

The software process model framework is specific to the project. Thus, it is essential to select the software process model according to the software which is to be developed.

The software project is considered efficient if the process model is selected according to the requirements. It is also essential to consider time and cost while choosing a process model as cost and/ or time constraints play an important role in software development.

The basic characteristics required to select the process model are project type and associated risks, requirements of the project, and the users.

One of the key features of selecting a process model is to understand the project in terms of size, complexity, funds available, and so on. In addition, the risks which are associated with the project should also be considered. Note that only a few process models emphasize risk assessment. Various other issues related to the project and the risks are listed in Table.

Table Selections on the Basis of the Project Type and Associated Risks

Project Type and Associated Risks	Waterfall	Prototype	Spiral	RAD	Formal Methods
Reliability requirements	No	No	Yes	No	Yes
Stable funds	Yes	Yes	No	Yes	Yes
Reuse components	No	Yes	Yes	Yes	Yes
Tight project schedule	No	Yes	Yes	Yes	No
Scarcity of resources	No	Yes	Yes	No	No

The most essential feature of any process model is to understand the requirements of the project. In case the requirements are not clearly defined by the user or poorly understood by the developer, the developed software leads to ineffective systems. Thus, the requirements of the software should be clearly understood before selecting any process model. Various other issues related to the requirements are listed in Table.

Table Selection on the Basis of the Requirements of the Project

Requirements of the Project	Waterfall	Prototype	Spiral	RAD	Formal Methods
Requirements are defined early in SDLC	Yes	No	No	Yes	No
Requirements are easily defined and understandable	Yes	No	No	Yes	Yes
Requirements are changed frequently	No	Yes	Yes	No	Yes
Requirements indicate a complex System	No	Yes	Yes	No	No

Software is developed for the users. Hence, the users should be consulted while selecting the process model. The comprehensibility of the project increases if users are involved in selecting the process model. It is possible that a user is aware of the requirements or has a rough idea of the requirements. It is also possible that the user wants the project to be developed in a sequential manner or an incremental manner (where a part is delivered to the user for use). Various other issues related to the user's satisfaction are listed in Table.

Table Selection on the Basis of the Users

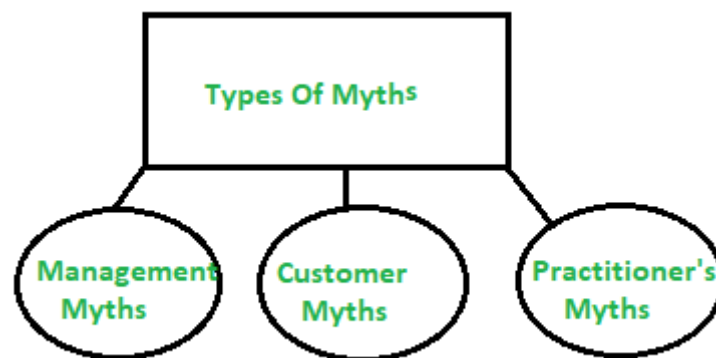
User Involvement	Waterfall	Prototype	Spiral	RAD	Formal Methods
Requires Limited User Involvement	Yes	No	Yes	No	Yes
User participation in all phases	No	Yes	No	Yes	No
No experience of participating in similar projects	No	Yes	Yes	No	Yes

Parameter	Process Model→	Waterfall Model	Incremental Model	Prototype Model	Rad Model	Spiral Model	Agile Model	Xp programming
Clear Requirement Specifications		Initial level	Initial level	At medium level	Initial level	Initial level	Change incrementally	Initial level
Feedback from user		No	No	Yes	No	No	No	Yes
Speed to change		Low	High	Medium	No	High	High	High
Predictability		Low	Low	High	Low	Medium	High	High
Risk identification		At initial level	No	No	No	Yes	Yes	Yes
Practically implementation		No	Low	Medium	No	Medium	High	High
Loom		Systematic sequence	Iterative sequence	Priority on customer feedback	Use readymade component	Identification of risk at each stage	Highly customer satisfaction and incremental development[09]	Customer satisfaction and incremental development
Any variation done		Yes-v model	No	No	No	Yes-win win spiral[6]	No	No
Understandability		Simple	Intermediate	Intermediate	Intermediate	Hard	Much complex	Intermediate
Precondition		Requirement clearly defined	Core product should clearly define	Clear idea of Quick Design	Clean idea of Reuse component	No	No	No
Usability		Basic	Medium	High	Medium	Medium	Most use now a days	medium
Customer priority		Nil	Nil	Intermediate	Nil	Intermediate	High	Intermediate
Industry approach		Basic	Basic	Medium	Medium	Medium	High	Medium
Cost		Low	Low	High	very high	Expensive	Much Expensive	High
Resource organization		Yes	Yes	Yes	Yes	No	No	Yes
Elasticity		No	No	Yes	Yes	No	Very high	Medium

Brief description about Software Myths

Software Myths:

Most, experienced experts have seen myths or superstitions (false beliefs or interpretations) or misleading attitudes (naked users) which creates major problems for management and technical people. The types of software-related myths are listed below.



`Types of Software Myths

(i) Management Myths:

Myth 1:

We have all the standards and procedures available for software development.

Fact:

- Software experts do not know all the requirements for the software development.
- And all existing processes are incomplete as new software development is based on new and different problem.

Myth 2:

The addition of the latest hardware programs will improve the software development.

Fact:

- The role of the latest hardware is not very high on standard software development; instead (CASE) Engineering tools help

the computer, they are more important than hardware to produce quality and productivity.

- Hence, the hardware resources are misused.

Myth 3:

- With the addition of more people and program planners to Software development can help meet project deadlines (If lagging behind).

Fact:

- If software is late, adding more people will merely make the problem worse. This is because the people already working on the project now need to spend time educating the newcomers, and are thus taken away from their work. The newcomers are also far less productive than the existing software engineers, and so the work put into training them to work on the software does not immediately meet with an appropriate reduction in work.

(ii) Customer Myths:

The customer can be the direct users of the software, the technical team, marketing / sales department, or other company. Customer has myths leading to false expectations (customer) & that's why you create dissatisfaction with the developer.

Myth 1:

A general statement of intent is enough to start writing plans (software development) and details of objectives can be done over time.

Fact:

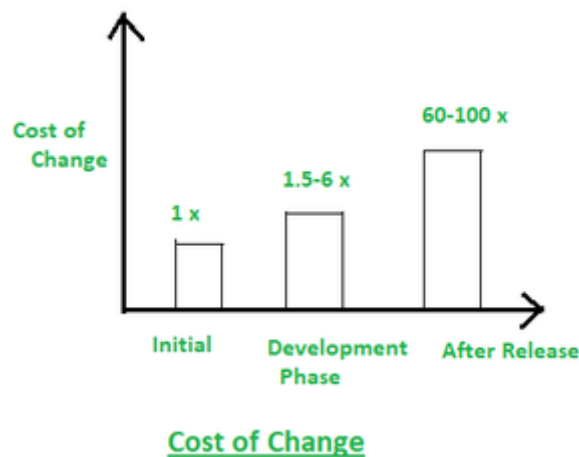
- Official and detailed description of the database function, ethical performance, communication, structural issues and the verification process are important.
- Unambiguous requirements (usually derived iteratively) are developed only through effective and continuous communication between customer and developer.

Myth 2:

Software requirements continually change, but change can be easily accommodated because software is flexible

Fact:

- It is true that software requirements change, but the impact of change varies with the time at which it is introduced. When requirements changes are requested early (before design or code has been started), the cost impact is relatively small. However, as time passes, the cost impact grows rapidly—resources have been committed, a design framework has been established, and change can cause upheaval that requires additional resources and major design modification.



Cost of Change
Different Stages of Myths

(iii) Practitioner's Myths:

Myths 1:

They believe that their work has been completed with the writing of the plan.

Fact:

- It is true that every 60-80% effort goes into the maintenance phase (as of the latter software release). Efforts are required, where the product is available first delivered to customers.

Myths 2:

There is no other way to achieve system quality, until it is “running”.

Fact:

- Systematic review of project technology is the quality of effective software verification method. These updates are quality filters and more accessible than test.

Myth 3:

An operating system is the only product that can be successfully exported project.

Fact:

- A working system is not enough, the right document brochures and booklets are also required to provide guidance & software support.

Myth 4:

Engineering software will enable us to build powerful and unnecessary document & always delay us.

Fact:

- Software engineering is not about creating documents. It is about creating a quality product. Better quality leads to reduced rework. And reduced rework results in faster delivery times